# Power and Heat Management in Embedded Systems

George Joseph Elassal
University of California, Irvine
Samueli School of Engineering
October 17th 2025

**Abstract**

Performance demands have increased as time moves forward and with it so has power and heat dissipation. As a result of this, tech industries have become increasingly barred from increasing performance due to power and heat issues. This problem is further magnified in the field of embedded systems where these devices are often powered by portable batteries with limited power. Embedded systems must be able to conserve power without giving up too much performance as to prolong device lifetime. For example, phone companies increase the quality of service by increasing the battery life of their devices, and yet they do not sacrifice performance compared to the former generations. Many researchers are currently working on pioneering methods based on the architecture of embedded systems to further lower power consumption/heat dissipation while keeping in mind performance tradeoffs. We have seen bus encoding, clock gating, cache power management, and more. In this paper we will cover some of the older well known technologies as well as diving into the newer methods such as dynamic voltage and frequency scaling and thermal task scheduling.

## 1 Introduction

First off, it is important to note the differences between embedded systems and general purpose computer systems. An embedded system is a specialized computer that is built into a larger device to perform a dedicated function. On the other hand general purpose computers such as desktops and laptops can run many types of programs and their system is built differently to handle that. Because of embedded systems more specific goals they can be specifically optimized to perform that job as efficiently as possible. Embedded systems are found everywhere; in smartphones, cars, medical devices, industrial machines, and even smart home appliances. These characteristics directly lead to why Power and Heat Management are major design challenges. Embedded systems have to deal with time critical computations. This means that haphazardly slowing down computation speed in order to lessen power and heat consumption is not viable. Missed deadlines can lead to failures or safety risks. For example, in a car's braking system or a pacemaker, even a small delay could cause serious consequences. Because of this performance requirements must be taken into account when optimizing power usage and heat generation. Since many embedded systems run off of batteries, there is an obvious need to create power efficiency to increase the lifetime of these devices. The importance of that need is seen when one looks at the lifetime of phones, speakers, and even the mileage of electric cars. Quality of service is greatly dependent on device lifetime. This power constraint from the batteries also restricts the complexity of the hardware that can be used. Cost constraints are also a big deal since embedded systems are usually manufactured in huge quantities, so adding small components to combat heat/power issues can cause huge cost increases. Even if cost was not a

problem, adding components and cooling solutions could still cause issues due to the fact that embedded systems are usually constrained by their size. They usually have to fit in small places like inside larger devices or in  tight places such as earbuds or glucose monitors. There literally might not be enough space for adding physical solutions. To add onto that, since embedded systems are so small, heat actually becomes a massive problem since excess heat can easily reduce system reliability or cause failures. In summary, power and heat management is a complex and critical issue when designing embedded systems.

### 1.1 Focus

In this paper we discuss what engineers look for when measuring performance relative to power and heat, how they quantify the performance, heat dissipation, and the energy usage. We will discuss how engineers decide whether performance tradeoffs are too large and what techniques they use to lower power consumption and heat generation. There are many older technologies that will be discussed as well as what seems to be the most important newer technology which is known as Dynamic Voltage and Frequency Scaling (DVFS).

## 2 Survey of Techniques

### 2.1 DPM

Moving onto one of the more basic techniques, there is dynamic power management (DPM). This is a key strategy for reducing power dissipation in embedded systems, and was the precursor to DVFS [3]. The basic idea is to selectively slow down or shut off components that aren’t being used. For example, DPM would entail putting a processor

or a memory module into a low-power state when it's idle. One of the simplest DPM methods is the time-out policy, where a component turns off after being inactive for a set amount of time. The efficiency of this system is based on how well the system can time the low and high power states. Turning power off too late and having to turn it back on again right after will ironically make the entire system less efficient. So, over time, researchers have improved on this by using predictive models that anticipate when components will be needed again based on past activity. More recently, the field has moved toward mathematical optimization approaches, such as the Markov Decision Processes which model the power system probabilistically and find the best balance between power savings and performance. These models can even handle continuous-time and more complex, real-world system behaviours, making them much more effective in modern embedded systems [3].

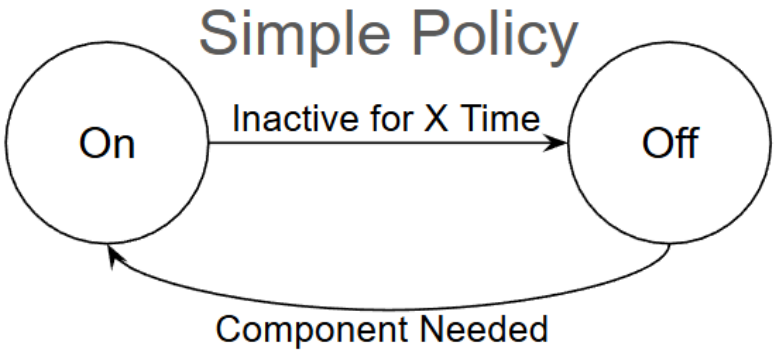


**Figure 1 DPM Simple Policy**

### 2.2 Low Power Bus Encoding

Next we have Low Power Bus Encoding, which is one of the main techniques used to reduce power dissipation during data transfer. In an embedded system, buses connecting the processor and memory handle heavy data traffic and this in turn can consume a lot of energy because every bit transition uses power. This is the problem

that bus encoding aims to solve. The main goal is to reduce the amount of switching buses have to go through or in other words the hamming distance. There are multiple ways to go about low power bus encoding [3].

### 2.2.1 Gray Code

One method is Gray Code. This is a design implementation that allows for only one bit change between consecutive addresses, making it ideal for sequential access. For example, when counting in binary, a transition from 0111 to 1000 causes four bits to flip, but in Gray Code, the transition from 0110 to 1100 changes only one bit. This significantly reduces the number of transitions and, in turn, the power consumed. Since things like program instructions usually are stored in order next to each other in memory, having a system that optimizes sequential access can have great effects on the efficiency of the whole system [3].

### 2.2.2 Bus-Invert Encoding

Next, Bus-Invert Encoding works by comparing each new data word to the previous one before transmission. If more than half of the bits would change, the system inverts all the bits and sends an additional “invert” flag. This guarantees that no more than half of the bits change during any transfer, effectively lowering switching activity [3].

### 2.2.3 T0 (Transition) Encoding

Transition Encoding is especially effective for sequential address patterns. Instead of repeatedly sending new address values, it keeps the bus lines constant and transmits only a small control signal to indicate an increment. This approach drastically reduces transitions on the bus. Again the point of all these techniques is simply to reduce the

bus lines from switching so much. The decrease of switching decreases power usage and heat generation accordingly [3].

### 2.3 Low Power System Synthesis

The next technique to discuss is Low Power System Synthesis. This is where engineers decide which parts of a system should run on hardware and which parts should run on software. This is less of a technique and more of thought-process that goes into creating embedded systems. To run a function in software, means the task is performed by a processor executing lines of code or in other words instructions stored in memory. When a function is said to run on hardware, it means that the same function is carried out by a dedicated circuit. For example, in the instance where a system needs to encrypt data, the software implementation has the CPU run a program that performs the encryption algorithm step by step. It's flexible and the engineers can change the algorithm easily by updating the code. Though this comes at the cost of performance and power consumption because of the fact that the CPU is doing general-purpose work. It isn't optimized to do the encryption specifically. On the other hand, in Hardware implementation the encryption algorithm is "hard-wired" into a custom circuit. It's similar to how there are specific adder and counting circuits on each board. Here It can run much faster and use less power because it's built specifically for that task. By optimizing tasks that are known to be used very often engineers can lower the power consumption and heat generation of the whole system by optimizing for some specific often used functions. This sort of implementation does have some problems though, as it is much less flexible. If an engineer wanted to change the algorithm, they would have to redesign the whole circuit. So there is an apparent tradeoff between repurpose-ability

and efficiency. Engineers must be very selective about which functions they choose to implement into the hardware to avoid this problem [3].

## 3 Dynamic Voltage and Frequency Scaling

Moving on, this section analyzes the main technique which is DVFS; it has quickly become one the main and most important power and heat management systems. In the most simple terms the mechanics are as such; the system automatically and dynamically changes the voltage and clock frequency being administered to the system depending on how much power the system actually needs. Since Power is proportional to the load capacitance, frequency, and voltage there is an obvious benefit of lowering the frequency and voltage. Using this down scaling to lower the power consumption has the added effect of lowering the heat generation as well. The complexity of DVFS comes as a result of the performance of the device being directly affected by the frequency of the clock. So, the real challenge, and what most of this paper focuses on, is how to decide when and by how much to lower the voltage and frequency without sacrificing too much performance.

### 3.1 RT-TRM [2]

Researchers Lee et al. proposed a solution to this problem with their Real-Time Thermal Aware Resource Management Framework or RT TRM. They found that many other researchers before them overlooked ambient temperature as a major factor in their heat management systems. Lee et al. recognized that the two main sources of heat in an embedded system come from not only the Task-level power dissipation but also the Dynamically Changing Ambient Temperature. They ran some experiments where they

looked at the correlation between ambient temperature and computational power and found a significant inverse relationship. For example, a 15 degree celsius increase in ambient temperature saw a 29% drop in computational power on their systems [2]. It is worth emphasizing that just going from season to season the temperature can change around 28 degrees celsius give or take. They even kept track of an embedded system on a car and saw that on a single drive the ambient temperature was increased by 180% [2]. The sort of slow downs these temperature changes can cause might make running a task actually overheat the system. So their motivation turned into how they could implement a heat management system that took into account both sources of heat whilst still maintaining maximum achievable performance.

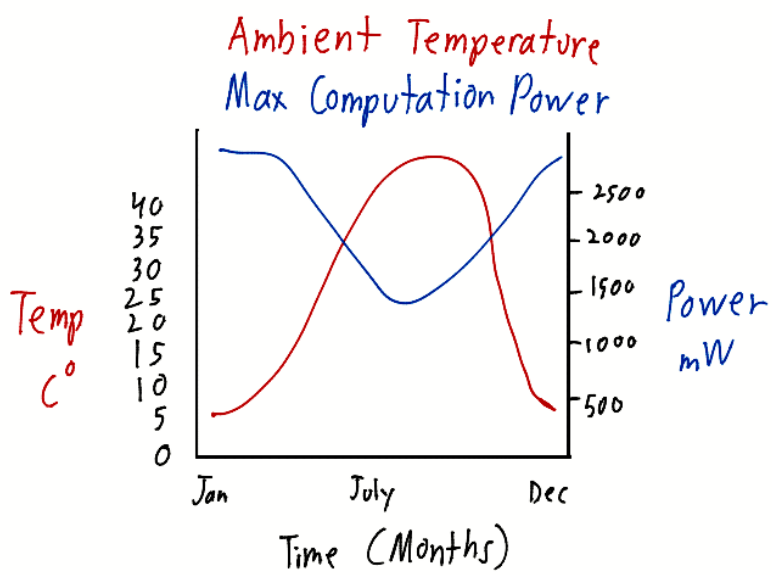


**Figure 2 Ambient Temperature vs Max Computation Power**

**(Results reproduced from data in Y. Lee et al., 2018 [2].)**

### 3.1.1 Thermal Modeling

The first step of their process was to create their thermal model. Here, the authors start by defining the thermal feasibility conditions for scheduling under DVFS. The main idea

is that every task set eventually reaches a steady-state temperature that's independent of how it's scheduled. So to keep the system thermally safe, they enforce the condition that this steady temperature $T(\infty,\tau)$ must remain below the thermal constraint Tmax. This condition translates into a power constraint. The power dissipated by all tasks, $PD(\tau)$, must not exceed the power bound PB(Tamb), which can change depending on the ambient temperature. $PD(\tau)$ represents the total power dissipation across all tasks, proportional to Voltage, frequency, and task utilization. Task utilization here is the sum of the activity factor multiplied by the execution time which is a function of the set frequency over the period of each task. PB(Tamb) is the upper power bound. It defines the maximum allowable power before the chip overheats, and it is derived from the thermal resistance and the temperature difference between the chip and its environment. If the parameters for Voltage frequency and period are set in a way that total power dissipation across all tasks is less than the calculated upper power bound then heat limits should never be passed. Finally, these thermal and power constraints are integrated into a classic real-time schedulability condition.The sum of utilizations, adjusted by thermal effects, must remain below one [2]. Or in other words, the system must not reach over 100% utilization. This ensures that all tasks meet their deadlines as well. Though they realized that a job schedule can affect the transient temperature. Because of this, they figured that each job has minimum idle time required depending on the ambient temperature and so they included it in their feasibility condition (cf. figure 3).

### 3.1.2 Optimization Problem

When they combine these two conditions they can create an optimization problem that

by solving will give them the most optimized parameters for voltage, frequency, and period of their DVFS system. Since voltage is a function of the frequency there are only two main variables to look at; frequency and the period for tasks. In this optimization, the goal isn't just to crank up the clock speed, it's to maximize overall system performance while keeping the system cool and on schedule. The equation is maximizing the weighted sum of task rates, which in essence means running important tasks as often as possible (cf. figure 3). To do that, the system adjusts both the clock frequency and each task's period, but it must still obey the two main constraints. The total power can't exceed the thermal bound, and all tasks must finish before their deadlines. So the algorithm finds the sweet spot between performance, power, and temperature. It should be noted that in practice, a processor can't just choose from infinite frequencies or periods. There is a discrete set of supported levels, known as DVFS states. Each state corresponds to a specific voltage and frequency pair that the hardware can safely use. And in this case there are also set periods as well. So the system is choosing from the best available frequencies and periods to optimize the system.

$$
\begin{aligned}
&\underset{f,p_i}{\text{maximize}} \sum_{\tau_i} w_i \cdot \frac{1}{p_i} \\
&\text{s.t.} \quad \overline{\text{C1}}\text{: } \mathrm{PD}(\tau) = V^2 \cdot f \cdot \sum_{\tau_i} \alpha_i \cdot \frac{e_i(f)}{p_i} \leq \mathrm{PB}(\mathrm{T_{amb}}) \\
&\qquad \overline{\text{C2}}\text{: } \sum_{\tau_i} \frac{e_i(f) + I_i^{\min}(T_{\text{amb}})}{p_i} \leq 1 \\
&\qquad f \in [f_{\min}, \ldots, f_{\max}]\,. \\
&\qquad \forall \tau_i \quad p_i^{\min} \leq p_i \leq p_i^{\max}.
\end{aligned}
$$

**Figure 3 Optimization Formula**

**(Equations reproduced from Y. Lee et al., (2018) [2])**

### 3.1.3 Online Idle Time Scheduling

Referring back to Section 3.1.1, where it was mentioned that a job schedule can affect the transient temperature, the next challenge the researchers Lee et al. faced was the fact that, though they could get the steady temperature to be assured under the heat constraint, they needed to create a task schedule where no tasks overheated the system before it got to the steady temperature. For example, in the case where the temperature is already at Tmax and there are two jobs, one hot and one cold, to be scheduled, the order matters. A hot job being one that would increase the temperature of the system and a cold job being a job that is so easy to run that it actually cools down the system. If the hot job were to be scheduled first, then the system would overheat. On the other hand, if the cold job was scheduled first, then the system would theoretically be able to stay under Tmax because the system was allowed to cool down before running the hot job. Now, what the researchers decided to do to fix this sort of problem is to schedule idle times in order to cool down the system before running jobs that would otherwise overheat the system. To do that, first they need to calculate the safe temperature required by the system before running a hot task, and then they must calculate the idle time necessary to reach that safe temperature. Now they want to be able to do this with pre-emption allowed, where a task can be split into multiple subtasks so that idle times can be inserted in between the subtasks. Initially, as the amount of subtasks the task is split into increases, there are more idle opportunities, the CPU can cool down or save power more often, and energy efficiency improves. Unfortunately, there are lessening returns, and after a certain point each new idle period becomes too short to matter, overhead starts to dominate, and total energy savings flatten or even

reverse. To find where adding more idle periods becomes not worth the switching penalties, they take the derivative of this energy-efficiency potential function with respect to the number of subtasks, and they find where the rate of change is less than or equal to the pre-emption cost. Now, using their models for safe temperature, necessary idle time to reach the safe temperature, and optimized amount of subtasks, they can calculate the minimum idle time required for each task. So now, using the Earliest Deadline First scheduling policy and making sure each task has its minimum required idle time, they can guarantee both timing and heat constraints, and this is also how they get the real-time scheduling condition mentioned in Section 3.1.1. Notably, updating idle time calculations only when DVFS parameters change is good enough to keep the system working efficiently [2]. Though, there is one more problem.

### 3.1.4 Slack Dispersal

They found that only using Static Idle Time Allocation was inefficient and caused lots of unnecessary pre-emption. Lee et al. proposed an online scheduling system that was able to dynamically allocate idle time by reclaiming slack and distributing the idle time to tasks that used more power. The amount of slack given to a task is calculated by how much idle time is minimally required plus how “power hungry” the task is and all proportional to how much slack is actually available.

### 3.1.5 Results

To test these results, they ran different heat management systems, including their own, on automotive and infotainment applications and compared the results. They had two simple Earliest Deadline First systems. The first system was EDF-A, which ran at 1 GHz. It had a 100% task rate, but it kept overheating. It violated the thermal limit 76.7%

of the time, reaching a max of 71.5 °C [2]. The second system, EDF-W, on the other hand, ran at 0.8 GHz. It had a 63.5% task rate, which was much slower, but at least it always stayed within the thermal limit. It maxed out at 59.1 °C but underutilized resources [2]. RT-MTC is similar to RT-TRM but without Thermal Aware Scheduling. It dynamically lowered the frequency down to 0.4 GHz. It only had minor violations of around 3% of the time, while reaching a max of 60.5 °C. It had a task rate of 67.2% [2]. This was reaching middle ground, but it wasn't good enough. RT-TRM, the proposed solution, actually kept the frequency at max most of the time and instead adjusted task periods. By doing this, it achieved great results. It had no violations, a 79.4% task rate, which was 18% over RT-MTC, and had a low runtime overhead [2]. It offered better performance all around whilst never violating heat constraints. These results prove that using a real-time adaptive system can be efficient, have good performance, and be worth implementing.

**3.2 Power-Aware Design Techniques of Secure Multimode Embedded Systems** [1]

Not every implementation of DVFS is in real-time, but most DVFS implementations do at least have to store different possible states for the system to be in. Researches K. Jiang et al. worked on a system that calculated states offline and had limited runtime selection. This is important to the topic of this paper because it emphasizes how engineers can store DVFS/System states to combat power consumption with the limited memory space allowed. Not only that but they also tied security concerns into their performance vs heat trade off calculations. In their systems they ran a task that would detect hardware intrusions and they could increase the strength of this detector by increasing the rate at which this task ran. Though this has the obvious tradeoff of

increasing the power demand of the system because more tasks means more power required obviously. They also encrypt some of the messages being sent inside their system and they can strengthen the encryption by having multiple rounds of encryption. And yet again they had the same problem, increasing the rounds of encryption demands more power and in turn creates more heat. These security variables add to the complexity of the state of the system because now the system has to integrate the possible security states as well as the DVFS states.

### 3.2.1 Modes and Pareto Spaces

Because each set of tasks (or Mode) can have different configurations of quality of service, confidentiality, intrusion accuracy, and power/heat limits, a problem is created where there are simply way too many possible configurations to store. To fix this K. Jiang et al. decided to calculate pareto spaces offline before run time and store the best of the pareto spaces. A pareto space is just the set of optimal configurations that are strictly better than all the other configurations for a mode. Still, there isn't enough memory to store all the pareto spaces due to its complexity. For example, for 10 tasks there are 1024 possible modes. The memory is simply not big enough to store that much. So K. Jiang et al. only chooses pareto spaces for modes that they deem to be important. These are modes that run very often or supermodes which are modes that contain large amounts of tasks [1].

### 3.2.2 Offline vs Runtime Duties

The next question becomes, what should be done with these pareto spaces? First of all, offline they get a set of pareto spaces that relate to important modes and supermodes. Now, dynamically at run time, when a mode is set to run, the system checks if that

mode has a saved pareto space, and if it does, then it simply sets the configurations to match the pareto space. The pareto space being saved means that it is already calculated to meet all system requirements like power, security, and performance. They use the earliest deadline first scheduling policy, so that confirms utilization doesn't go over 100%. On the other hand, if the mode doesn't have a saved pareto space, the system chooses from a nearby super mode to match configuration. Since a supermode is a mode with a large amount of tasks, the system can trim unused tasks from that super mode and reallocate resources to improve quality of service while rechecking constraints. If the system cannot find any configuration that satisfies all constraints, it enters a safe fallback mode. In this mode, only the mandatory parts of tasks are executed, while optional QoS improvements are shut down. Security protections stay active, but performance gets reduced to prevent overload and thermal/power violations [1].

### 3.2.3 Results and Takeaways

The results from testing show that storing only a subset of carefully chosen Pareto spaces still allows the system to adapt effectively at runtime. Even when a mode was not stored, the system could derive a near-optimal configuration with only about a 3% performance loss. Runtime overhead was under one millisecond, making the approach practical. Compared to a simple greedy method, the Pareto-based scheduling improved quality of service by 40–80%, while still respecting thermal and security constraints. A great takeaway from the research done by K. Jiang et al. is that security cannot be ignored when dealing with heat and power management. They actually show that security tasks themselves can heat the system up. For example, if someone were to run

intrusion detection more often, or choose stronger encryption with more rounds, that would directly pull more CPU power and generate more heat. So security is part of the power–thermal tradeoff, not separate from it. Another important thing to note is that they figured out a smart way to manage all the possible configurations. Because every mode can run at different voltages, different encryption levels, different QoS, and different ID rates, the amount of possible configurations expands rapidly. Instead of storing everything, they only store the pareto-optimal ones. These are the ones that have the best tradeoffs, where improving one stat would always have a cost somewhere else. So they only save the useful configurations. The last takeaway is that because those pareto sets are computed offline, the system doesn't need to think hard at runtime. At runtime, it just looks up the configuration, or if the mode isn't stored, it derives one from a bigger supermode, and that makes switching really fast. Overall, the system balances real-time deadlines, security demands, and power/heat limits all at once. So it's not just trying to maximize QoS, it's making sure everything stays safe and within thermal limits while still meeting deadlines [1].

### 3.3 CPU-GPU-MEMORY DVFS [4]

The research done on CPU-GPU-Memory DVFS by S. Dey et al. may be the most important research covered in this paper. This is because it starts taking DVFS into modern applications. Modern embedded systems usually have multiple processors and a GPU. They're heterogeneous. The problem with the last two papers covered in sections 3.1 and 3.2 is that they were implementing DVFS on a single processor, but modern processors, as mentioned, have multiple. To adapt they would have to update their models. For example, in the RT_TRM case they would have to think about how the

heat from a CPU could directly heat up other nearby CPUs. They would have to add a whole new calculation into their thermal model. Now not only does S. Dey et al. dig into implementing DVFS across multiple CPUs and GPUs, but it also takes into consideration the memory. The researchers S. Dey et al. found that during memory intensive tasks the memory can account for about 20% of the total power usage [4] (cf. figure 4). This shows that applying DVFS to only a single CPU leaves a lot of room for power and heat savings over the whole system.

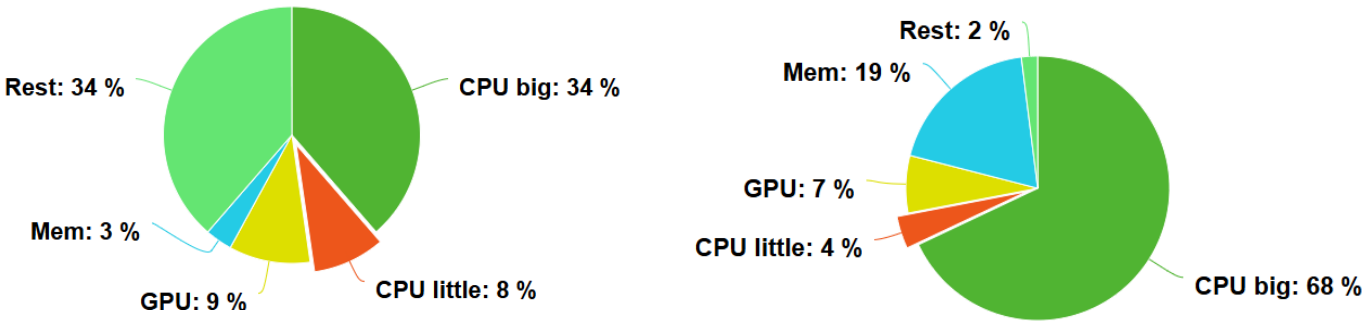


**Figure 4 Power Consumption by Hardware Components**

**(Results reproduced from data in S. Dey et al., (2022) [4])**

### 3.3.1 CGM DVFS Implementation

Before explaining their implementation, it is important to note that they were specifically working with mobile devices and their testing was done with different apps. Accordingly, they don't just randomly adjust frequencies like reinforcement-learning based systems do. They profile the app first. This means they run the app under different CPU, GPU, and memory frequency settings, and they record the performance, power, and thermal behavior for each one. Instead of testing every possible combination, which would be

counter-productive, they cluster DVFS levels that would produce similar results. So they reduce the search space down to just 216 configurations that actually matter instead of thousands upon thousands.  Then at runtime, once the app is running, the system wants a minimum of 60 fps, so it just picks the cheapest configuration that still gives 60 FPS and applies those DVFS settings. In this case the cheapest configuration is the configuration that takes the least amount of power. Because of this method, the system meets the performance target, but without wasting extra power, and in turn without overheating. This is very similar in spirit to the K. Jiang et al. paper, where they computed pareto-optimal configurations offline, stored them, and then selected from them at runtime. The key difference here is that this paper applies that same idea across multiple processors; big CPUs, little CPUs, GPUs, and memory instead of just one CPU. This sort of implementation is much closer to how modern embedded systems really behave [4].

### 3.3.2 Results and Conclusions

The researchers S. Dey et al. tested their implementation in a very practical sense. They used an Odroid XU4 board, which is fundamentally the same processor that used to be in Samsung Galaxy phones, so they used real mobile hardware, not simulation. Then instead of testing synthetic workloads only, they ran real apps such as face detection, YOLO object detection, live video rendering, YouTube-like streaming. Essentially, they used common mobile behaviors. For these apps, they measured performance using FPS and set a performance goal of 60 FPS.  Next, they profiled each app offline across 216 carefully selected DVFS configurations for the big CPUs, LITTLE CPUs, GPU, and memory. For each configuration they recorded power draw,

peak temperature, and the achieved performance. Then at runtime, the system just picks the configuration that hits the FPS target while using the least power. Finally, they compared their approach against the usual Linux governors and also against reinforcement learning based DVFS systems. Their method ended up using significantly less power while staying cooler and without dropping performance. Overall, the results were solid. When they tested this on real mobile applications, the CGM-DVFS approach cut power usage by about 26%, and reduced thermal load by around 21%, compared to existing DVFS systems. And importantly, it did this without sacrificing performance. It still held the 60 FPS target. In conclusion, the big takeaway here is that modern embedded systems are heterogeneous. If you only scale the CPU, you miss a huge portion of power usage. This paper shows that once you include GPU and memory DVFS into the control loop, you can get way better efficiency while still respecting performance and thermal limits.

**4 Conclusions and Future Directions**

The combination of these papers have left important takeaways. RT-TRM showed how thermal modeling is possible and how they can implement that into a real-time DVFS system. K. Jiang et al. showed how to do offline configuration calculation and use that for quick dynamic mode lookups during run time. They also showed how power and heat are not independent of security in embedded systems. Finally, S. Dey et al., with their CGM DVFS implementation, showed that modern embedded systems require DVFS to be applied across multiple components and not just on a single CPU. Together, these approaches reinforce the idea that effective power and heat management is not

just a local CPU-level problem, but instead is a multi-domain, global system-level design challenge.

### 4.1 Future Work

Looking toward the future, it seems that the emerging AI technologies will be used for real-time prediction of DVFS states. The CGM DVFS was tested against a machine learning model, and it was found that when the machine learning model was trained for a very long period of time, it could actually beat out the performance and power savings of the CGM model. Though, at lower training times, it proved to be less efficient. As AI becomes more and more streamlined and powerful, it seems that there will be a greater implementation of it into the thermal and schedule modeling for embedded systems. Ultimately, the involvement of AI will come in tandem with the work that has been done before it. Heterogeneity, thermal modeling, offline calculating, security concerns, and AI together define the future direction of power and heat management in embedded systems.

Bibliography

[1] K. Jiang, Y. Zhang, Q. Zhu, and M. A. Al Faruque, “Power-Aware Design Techniques of Secure Multimode Embedded Systems,” ACM Transactions on Design Automation of Electronic Systems (TODAES), vol. 21, no. 3, Feb. 2016, Art. no. 45, doi: 10.1145/2801152.

[2] Y. Lee, H. S. Chwa, K. G. Shin, and S. Wang, “Thermal-Aware Resource Management for Embedded Real-Time Systems,” IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 37, no. 11, pp. 2857–2868, Nov. 2018, doi: 10.1109/TCAD.2018.2857279.

[3] M. Pedram, “Power and Heat Management in Embedded Systems,” in Proceedings of the 2001 International Symposium on Low Power Electronics and Design (ISLPED ’01), Aug. 2001, pp. 1–5, doi: 10.1145/370155.370333.

[4] S. Dey, S. Isuwa, S. Saha, A. K. Singh, and K. McDonald-Maier, “CPU-GPU-Memory DVFS for Power-Efficient MPSoC in Mobile Cyber Physical Systems,” Future Internet, vol. 14, no. 3, Art. no. 91, Mar. 2022, doi:10.3390/fi14030091.